

LABORATORY MANUAL: EXPERIMENT

1. Objective

To design, implement, and analyze a C program that automatically computes the FIRST and FOLLOW sets for the non-terminal symbols of a given Context-Free Grammar (CFG). This builds the necessary mathematical framework required to construct predictive top-down LL(1) and bottom-up LR(1)/LALR(1) parsing tables.

2. Theoretical Background

During syntax analysis, the compiler checks if the tokens produced by the lexical analyzer satisfy the structural rules defined by the grammar. Computing FIRST and FOLLOW sets allows a deterministic parser to look ahead at incoming tokens and choose the correct production branch without backtracking.

- **FIRST(X)**: The set of all terminal symbols that can appear as the first symbol of a string derived from X. If X can derive an empty string (ϵ), then ϵ is also added to FIRST(X) (denoted as 'e' in our program environment).
- **FOLLOW(A)**: The set of all terminal symbols that can appear immediately to the right of the non-terminal A in some valid sentential derivation block. The special system end-marker symbol \$ is automatically assigned to the grammar's start symbol.

3. Mathematical Algorithms

A. Computation of FIRST Sets

1. Initialize FIRST(X) as an empty set for all non-terminal symbols X.
2. Iterate through the production rules until a full sweep across the entire grammar adds no new symbols to any set:
 - If the production is $X \rightarrow t\alpha$ (where t is a terminal), add t to FIRST(X).
 - If the production is $X \rightarrow e$ (epsilon), add e to FIRST(X).
 - If the production is $X \rightarrow Y\alpha$ (where Y is a non-terminal):
 - Add all symbols from FIRST(Y) (excluding e) directly into FIRST(X).
 - If FIRST(Y) contains e, proceed to evaluate the next adjacent token symbol in α . If all consecutive symbols in the derivation body can reduce to e, append e to FIRST(X).

B. Computation of FOLLOW Sets

1. Seed the start symbol S by adding the end marker \$ into FOLLOW(S). Initialize all other non-terminal sets as empty.
2. Iterate sweeps through all rules until sets stabilize completely without any changes:
 - For each production rule containing a non-terminal symbol on the right side, such as $A \rightarrow \alpha B \beta$:
 - Add all terminal items from FIRST(β) (excluding e) into FOLLOW(B).
 - If β can derive e (or if β does not exist because B sits at the absolute trailing boundary of the rule: $A \rightarrow \alpha B$), merge everything currently inside FOLLOW(A) straight into FOLLOW(B).

4. Source Code

Implement the solution below in standard C. This framework relies on structured fixed arrays and null-terminated string logic to track states without requiring external library configurations.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#define MAX_RULES 20
#define MAX_PROD 10
#define MAX_SYMBOLS 10

// Structure to represent a grammar production rule
typedef struct {
    char head;
    char body[MAX_PROD][MAX_SYMBOLS];
    int prod_count;
} GrammarRule;

GrammarRule grammar[MAX_RULES];
int rule_count = 0;

// Arrays to store FIRST and FOLLOW sets
char first_sets[26][MAX_SYMBOLS * 2];
char follow_sets[26][MAX_SYMBOLS * 2];
```

```

// Helper to check if a symbol is a Non-Terminal (Uppercase
letters)
bool is_non_terminal(char symbol) {
    return (symbol >= 'A' && symbol <= 'Z');
}

// Helper to add a terminal to a set uniquely
bool add_to_set(char set[], char val) {
    int len = strlen(set);
    for (int i = 0; i < len; i++) {
        if (set[i] == val) return false;
    }
    set[len] = val;
    set[len + 1] = '\0';
    return true;
}

// Helper to merge set2 into set1 (ignoring epsilon 'e')
bool merge_sets(char set1[], char set2[], bool
ignore_epsilon) {
    bool changed = false;
    int len2 = strlen(set2);
    for (int i = 0; i < len2; i++) {
        if (ignore_epsilon && set2[i] == 'e') continue;
        if (add_to_set(set1, set2[i])) {
            changed = true;
        }
    }
    return changed;
}

// Helper to check if a set contains epsilon ('e')
bool has_epsilon(char set[]) {
    for (int i = 0; set[i] != '\0'; i++) {
        if (set[i] == 'e') return true;
    }
    return false;
}

// Find rule index for a given non-terminal character
int find_rule_index(char head) {

```

```

    for (int i = 0; i < rule_count; i++) {
        if (grammar[i].head == head) return i;
    }
    return -1;
}

void compute_first() {
    bool changed = true;
    while (changed) {
        changed = false;

        for (int i = 0; i < rule_count; i++) {
            char head = grammar[i].head;
            int idx = head - 'A';

            for (int j = 0; j < grammar[i].prod_count; j++)
            {
                char *prod = grammar[i].body[j];
                int prod_len = strlen(prod);

                if (prod[0] == 'e') { // Epsilon rule
                    if (add_to_set(first_sets[idx], 'e'))
changed = true;
                } else {
                    bool next_symbol = true;
                    for (int k = 0; k < prod_len &&
next_symbol; k++) {
                        char sym = prod[k];
                        if (!is_non_terminal(sym)) {
                            if (add_to_set(first_sets[idx],
sym)) changed = true;

                            next_symbol = false;
                        } else {
                            int sym_idx = sym - 'A';
                            if (merge_sets(first_sets[idx],
first_sets[sym_idx], true)) changed = true;
                            if
(!has_epsilon(first_sets[sym_idx])) {
                                next_symbol = false;
                            }
                        }
                    }
                }
            }
        }
    }
}

```

[illegible]


```

// Add rule for S
grammar[rule_count].head = 'S';
strcpy(grammar[rule_count].body[0], "CB");
grammar[rule_count].prod_count = 1;
rule_count++;

// Add rule for C
grammar[rule_count].head = 'C';
strcpy(grammar[rule_count].body[0], "cC");
strcpy(grammar[rule_count].body[1], "d");
grammar[rule_count].prod_count = 2;
rule_count++;

// Add rule for B
grammar[rule_count].head = 'B';
strcpy(grammar[rule_count].body[0], "bC");
//strcpy(grammar[rule_count].body[1], "e");
grammar[rule_count].prod_count = 2;
rule_count++;

char start_symbol = 'S';

// Computations
compute_first();
compute_follow(start_symbol);

// Print Results
printf("--- FIRST SETS ---\n");
for (int i = 0; i < rule_count; i++) {
    char head = grammar[i].head;
    printf("FIRST(%C) = { ", head);
    char *s = first_sets[head - 'A'];
    for (int j = 0; s[j] != '\0'; j++) {
        printf("%c%s", s[j], (s[j+1] != '\0') ? ", " :
"");
    }
    printf(" }\n");
}

printf("\n--- FOLLOW SETS ---\n");
for (int i = 0; i < rule_count; i++) {

```

```

        char head = grammar[i].head;
        printf("FOLLOW(%C) = { ", head);
        char *s = follow_sets[head - 'A'];
        for (int j = 0; s[j] != '\0'; j++) {
            printf("%c%s", s[j], (s[j+1] != '\0') ? ", " :
"");
        }
        printf(" }\n");
    }

    return 0;
}

```

5. Lab Assignments & Tasks

TASK A: Code Modification & Alternate Grammar Implementation

Modify the hardcoded grammar structural matrix definitions inside the main() function of the provided C script to implement and evaluate the following standard algebraic production grammar:

$$\begin{aligned}
 E &\rightarrow T E' \\
 E' &\rightarrow + T E' \mid e \\
 T &\rightarrow F T' \\
 T' &\rightarrow * F T' \mid e \\
 F &\rightarrow (E) \mid id
 \end{aligned}$$

Hint: Represent non-terminals with single alphabetic uppercase variables (e.g., use E for E, A for E', T for T, B for T', and F for F) and use i to denote the identifier terminal string id.